

Week_1예습과제

CH01 파이썬 기반의 머신러닝과 생태계 이해

1. 머신러닝의 개념

머신러닝 :

애플리케이션을 수정하지 않고도 데이터 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법.

머신러닝 알고리즘은 데이터 기반으로 통계적인 신뢰도를 강화하고 예측 오류를 최소화하기 위한 다양한 수학적 기법을 적용해 데이터 내의 패턴을 스스로 인지하고 신뢰도 있는 예측 결과를 도출.

머신러닝의 분류:

지도학습/비지도학습/강화학습

- 머신러닝은 데이터 의존적이므로 좋은 품질의 데이터가 필요 → 데이터를 이해하고 효율적으로 가공, 처리, 추출해 최적의 데이터를 기반으로 알고리즘을 구동할 수 있도록 준비하는 능력이 중요.

파이썬 vs R : 개발 전문 프로그램 vs 통계 전용 프로그램

파이썬의 특성 : 쉽고 뛰어난 개발 생산성, 많은 라이브러리, 확장성 유연성 호환성으로 인한 광범위한 사용, 다양한 애플리케이션 개발 가능, 기업 환경으로의 확산 가능

2. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

머신러닝 패키지	사이킷런
행렬/선형대수/통계패키지	넘파이 - 행렬 기반 데이터 처리에 특화
데이터핸들링	판다스 - 2차원 데이터 처리에 특화
시각화	맷플롯립, 시본

- 넘파이와 판다스는 사이킷런의 머신러닝알고리즘에 입력하기 위한 데이터의 처리(추출,가공,변형, 원하는 차원 배열로의 변환을 포함해 머신러닝 알고리즘 처리 결과에 대한 다양한 가공 등)를 담당함.

3. 넘파이(Numpy))

넘파이 :

선형대수 기반의 프로그램을 쉽게 만들 수 있도록 지원하는 패키지. 빠른 배열 연산 속도, 다양한 데이터 핸들링 기능 제공. 많은 머신러닝 알고리즘이 넘파이 기반으로 작성돼 있으며 알고리즘의 입력, 출력데이터를 넘파이 배열 타입으로 사용. `import numpy (as np)`로 호출.

`ndarray` :

넘파이 기본 데이터 타입 , `ndarray` 내의 데이터타입은 `ndarray.dtype` 으로 확인할 수 있으며 같은 데이터 타입만 가능(더 큰 타입으로 형 변환)

`array()` : 다양한 인자 → `ndarray`타입

`ndarray.shape` : `ndarray`의 차원과 크기를 튜플 형태로

`arrange(stop인자 값)` : 0부터 (stop인자-1)까지 순차적으로 생성

`zeros/ones(튜플형태)` : 해당 튜플 형태로 모든 값을 0/1로 생성

`ndarray.reshape(튜플형태)` : `ndarray`를 특정 차원 및 크기로 변환, 이 때 로우 또는 칼럼 인자에 -1을 넣으면 고정된 칼럼 또는 로우 인자에 맞춰 자동으로 변환됨.

팬시 인덱싱 : 리스트나 `ndarray`로 인덱스 집합을 지정하면 해당 위치의 인덱스에 해당하는 `ndarray`를 반환하는 인덱싱 방식 ex. `array2 = array1[[0,1], 2]`

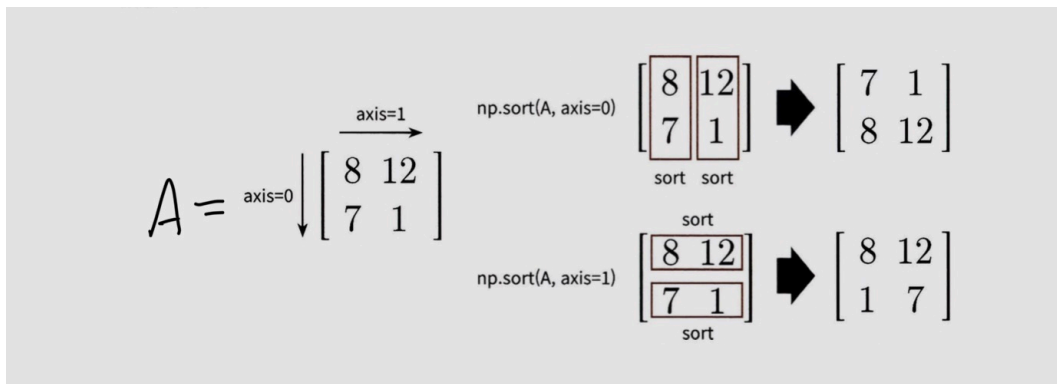
불린 인덱싱 : 조건필터링과 검색을 동시에, [] 내에 조건문을 그대로 기재해 true인 데이터들만 추출

행렬의 정렬 : 기본적으로 행렬 내 원소들을 오름차순으로 정렬

`np.sort()` vs `ndarray.sort()` : 원본 행렬 변경x vs 변경o

`sort()[::-1]` : 내림차순 정렬

2차원 정렬 :



`np.argsort()` : 정렬된 행렬의 원본 행렬 인덱스를 ndarray형으로 반환 → 반환 받을 변수 필요

넘파이를 이용한 선형대수 계산 :

`np.dot(A, B)` : 행렬 A,B의 내적(행렬곱)

`np.transpose(A)` : 행렬 A의 전치행렬

4. 데이터 핸들링-판다스

판다스 :

2차원 데이터를 효율적으로 가공/처리할 수 있는 다양하고 훌륭한 기능 제공, 파이썬 내부 데이터 뿐 아니라 CSV 등의 파일을 쉽게 DataFrame으로 변경해 데이터의 가공/분석을 편리하게 수행할 수 있게 해줌. `import pandas (as pd)`로 호출

DataFrame :

행, 열로 이루어진 2차원 데이터를 담는 데이터 구조체, series(칼럼이 하나 뿐인, index를 key값으로 갖는데이터구조체) 여러개로 이루어짐. index를 key값으로 맞춤.

DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환 :

`pd.DataFrame()` 인자로 2차원 리스트, 딕셔너리, ndarray 전달 시 변환됨. 반대로 `.values`로 ndarray 변환 가능하며 `.tolist()`나 `.to_dict()`로 역변환됨. 리스트와 ndarray는 값의 배열만 전달되나 딕셔너리는 Key가 칼럼명으로 자동 매핑됨.

DataFrame의 칼럼 데이터 세트 생성과 수정 :

df['새칼럼명'] = 값 형태로 지정 후 연산 결과나 값 할당으로 추가됨. 기존 칼럼명 선택 후 새로운 값이나 연산 결과 대입 시 갱신됨. 벡터화 연산 지원으로 반복문 없이 전체 행 일괄 수정 및 생성됨.

DataFrame 데이터 삭제 :

drop() 메서드 활용 시 axis=0은 행 삭제, axis=1은 칼럼 삭제됨. 기본적으로 삭제된 새 DataFrame 반환되며 원본 적용 시 inplace=True 지정 또는 재할당함. 삭제할 인덱스나 칼럼명을 리스트 형태로 전달하여 일괄 제거함.

Index 객체 :

DataFrame과 Series의 각 행을 제어하고 단일하게 식별하는 Key 값 객체임. 생성된 Index 값은 수정 불가하여 단순 참조 및 연산용으로만 쓰임. reset_index() 메서드로 기존 Index를 일반 칼럼으로 변환하고 정수형 Index로 재설정함.

데이터 셀렉션 및 필터링 :

명칭 기반인 .loc[]는 종료 범위까지 포함되고 위치 기반인 .iloc[]는 종료 직전 범위까지 추출됨. df[df['Age'] > 60] 구문처럼 조건에 부합하는 행만 추출하는 불리언 인덱싱 활용함. 복합 조건 결합 시 &와 | 연산자 사용하며 각 조건은 소괄호 ()로 감싸서 처리함.

정렬, Aggregation 함수, Groupby 적용 :

sort_values()의 by 옵션으로 기준 칼럼 지정하고 ascending=False로 내림차순 및 다중 정렬함. groupby() 실행 시 DataFrameGroupBy 객체 반환되며 이후 집계 함수 연결함. agg()에 딕셔너리 전달하여 칼럼별 다른 집계 연산 처리함.

결손 데이터 처리하기 :

df.isna().sum()으로 칼럼별 결측치 개수 파악함. df['칼럼'].fillna(대체값) 구문으로 NaN을 평균이나 최빈값 등 대표값으로 대체함. 결측치가 포함된 행이나 열을 정제할 때는 dropna() 메서드 적용함.

Apply Lambda 식으로 데이터 처리 :

apply() 메서드로 모든 요소에 사용자 정의 함수나 lambda 연산 일괄 적용함. lambda x: 식 구문으로 별도 def 선언 없이 한 줄로 데이터 변환함. 조건문 구성이나 별도 def 사용자

정의 함수 결합하여 복잡한 로직 처리함.